Tries

1. Implement Trie – 1

We need to implement a Trie (Prefix Tree) that supports the following operations:

- **insert(word)**: Insert a word into the Trie.
- **search(word)**: Return true if the exact word exists in the Trie.
- **startsWith(prefix)**: Return true if there is any word in the Trie that starts with the given prefix.

A Trie stores characters level by level, where each node represents a character. This helps in fast prefix-based searching.

Example:
If we insert `"apple"` and `"app"`:

- Searching `"apple"` → true
- Searching `"app"` → true
- Searching `"ap"` → false
- startsWith `"ap"` → true

Approach

Algorithm

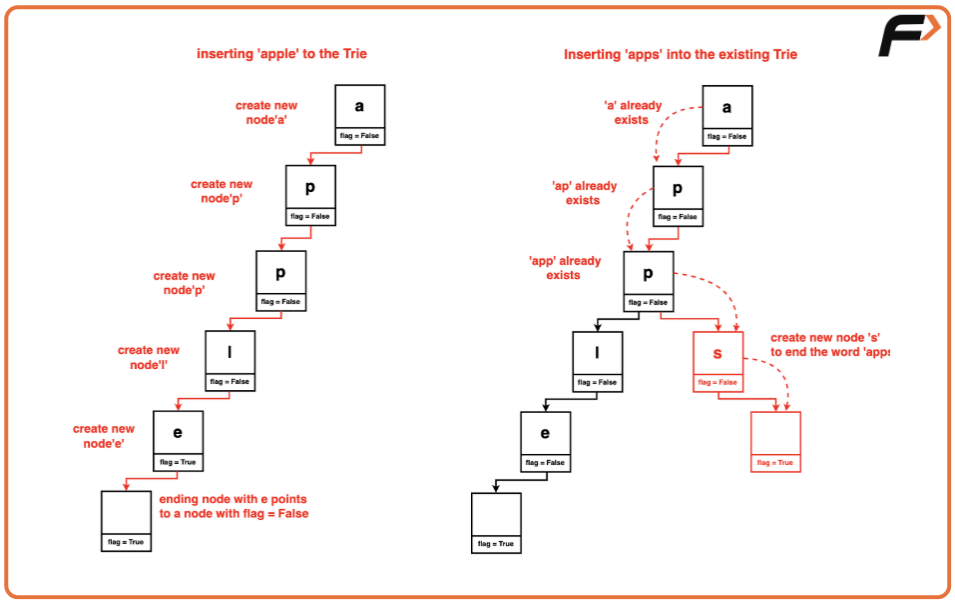
A Trie is made of nodes. Each node contains:

- An array of size 26 to store links for characters `'a'` to `'z'`
- A boolean flag to mark whether a word ends at this node

Insert operation

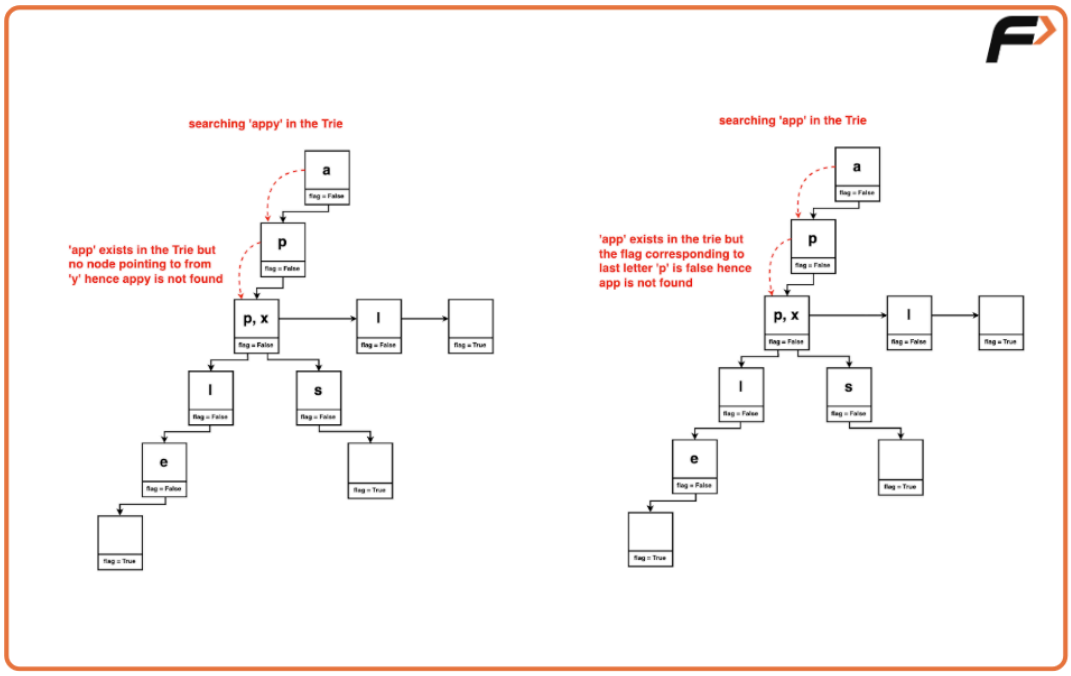
1. Start from the root node.

2. For each character in the word:
    - If the child node for that character does not exist, create it.
    - Move to the child node.
3. After inserting all characters, mark the last node as end of word.



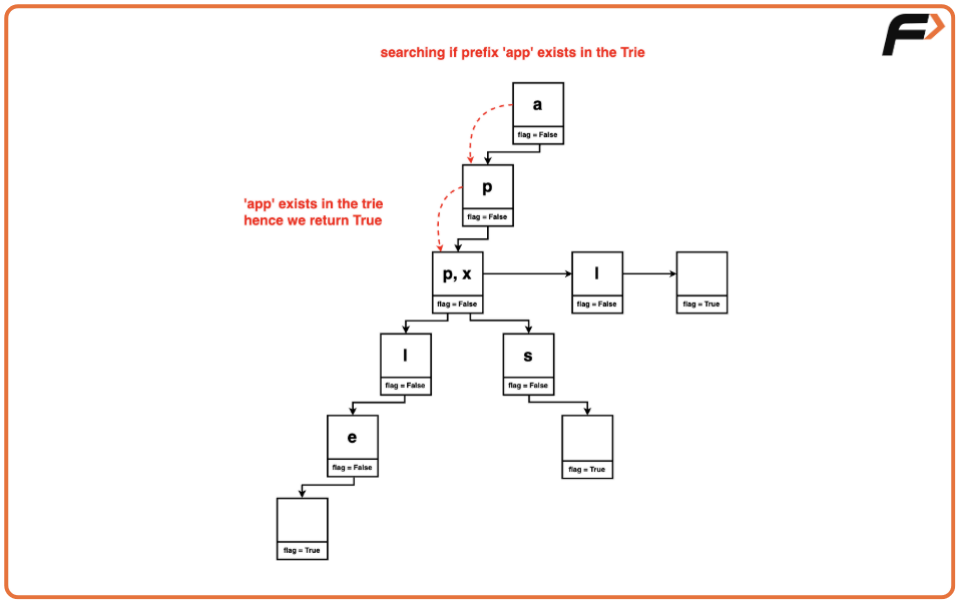
Search operation

1. Start from the root node.
2. For each character in the word:
    - If the child node does not exist, return false.
    - Move to the child node.
3. After all characters are processed, check if the current node is marked as end of word.



Prefix check (startsWith)

1. Start from the root node.
2. For each character in the prefix:
    - If the child node does not exist, return false.
    - Move to the child node.
3. If all characters are found, return true.



Code

```
#include <bits/stdc++.h>
using namespace std;

// Node structure for Trie
struct Node {
    Node* links[26] = {nullptr};
    bool flag = false;

    bool containsKey(char ch) {
        return links[ch - 'a'] != nullptr;
    }

    void put(char ch, Node* node) {
        links[ch - 'a'] = node;
    }

    Node* get(char ch) {
        return links[ch - 'a'];
    }
```

```
    void setEnd() {
        flag = true;
    }

    bool isEnd() {
        return flag;
    }
};

// Trie class
class Trie {
private:
    Node* root;

public:
    Trie() {
        root = new Node();
    }

    void insert(string word) {
        Node* node = root;
        for(char ch : word) {
            if(!node->containsKey(ch)) {
                node->put(ch, new Node());
            }
            node = node->get(ch);
        }
        node->setEnd();
    }

    bool search(string word) {
        Node* node = root;
        for(char ch : word) {
            if(!node->containsKey(ch)) {
                return false;
            }
            node = node->get(ch);
        }
```

```
        return node->isEnd();
    }

    bool startsWith(string prefix) {
        Node* node = root;
        for(char ch : prefix) {
            if(!node->containsKey(ch)) {
                return false;
            }
            node = node->get(ch);
        }
        return true;
    }
};

int main() {
    Trie trie;
    trie.insert("apple");
    cout << trie.search("apple") << endl;   // true
    cout << trie.search("app") << endl;     // false
    cout << trie.startsWith("app") << endl; // true
    trie.insert("app");
    cout << trie.search("app") << endl;     // true
    return 0;
}
```

Complexity Analysis

Time Complexity

- Insert: O(L), where L is the length of the word
- Search: O(L)
- startsWith: O(L)

This is because we process one character at a time.

Space Complexity

- O(N), where N is the total number of characters stored in the Trie
  Each character may create a new node in the Trie.

2. Implement Trie – II (with frequency counts)

This is the **extended Trie** version where, unlike Trie-I, we must **count occurrences** of words and prefixes and also **erase** words correctly.

What's new compared to Trie-I?

Each Trie node now maintains **two counters**:

1. **cntEndWith** →
   How many words **end exactly at this node**

2. **cntPrefix** →
   How many words **pass through this node** (i.e., have this prefix)

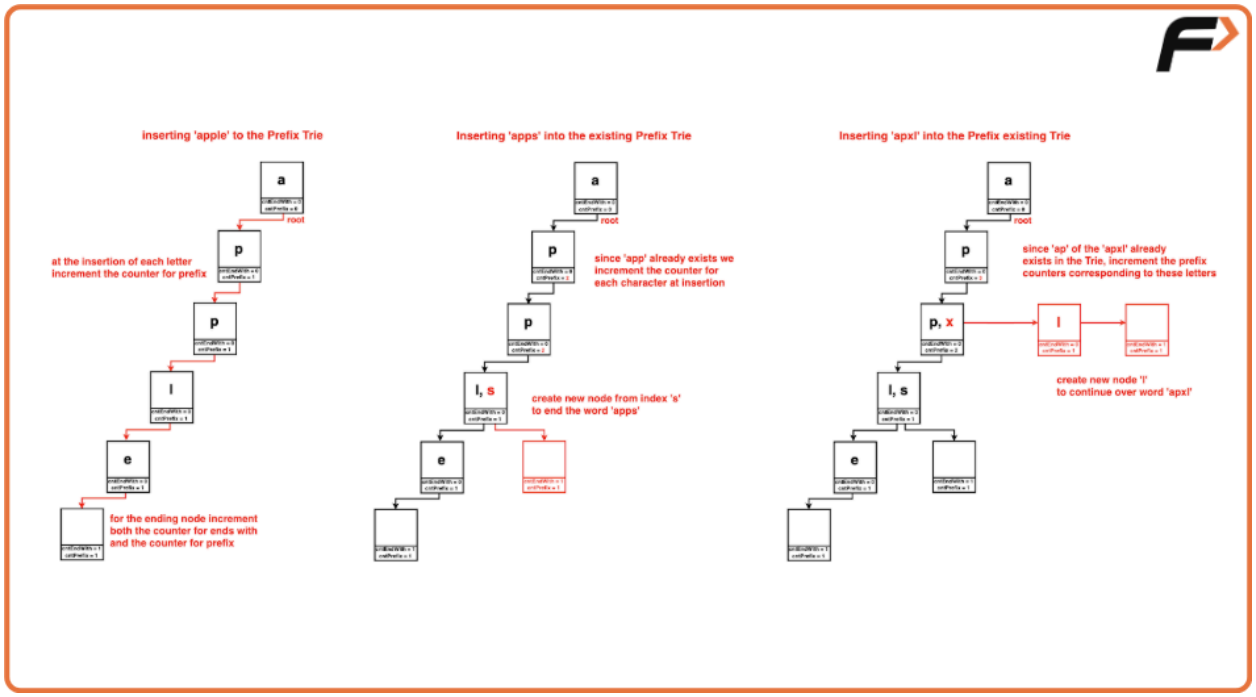
These counters allow us to:

- Count duplicate insertions
- Count how many words share a prefix
- Erase words safely without deleting unrelated prefixes

Core Operations Explained

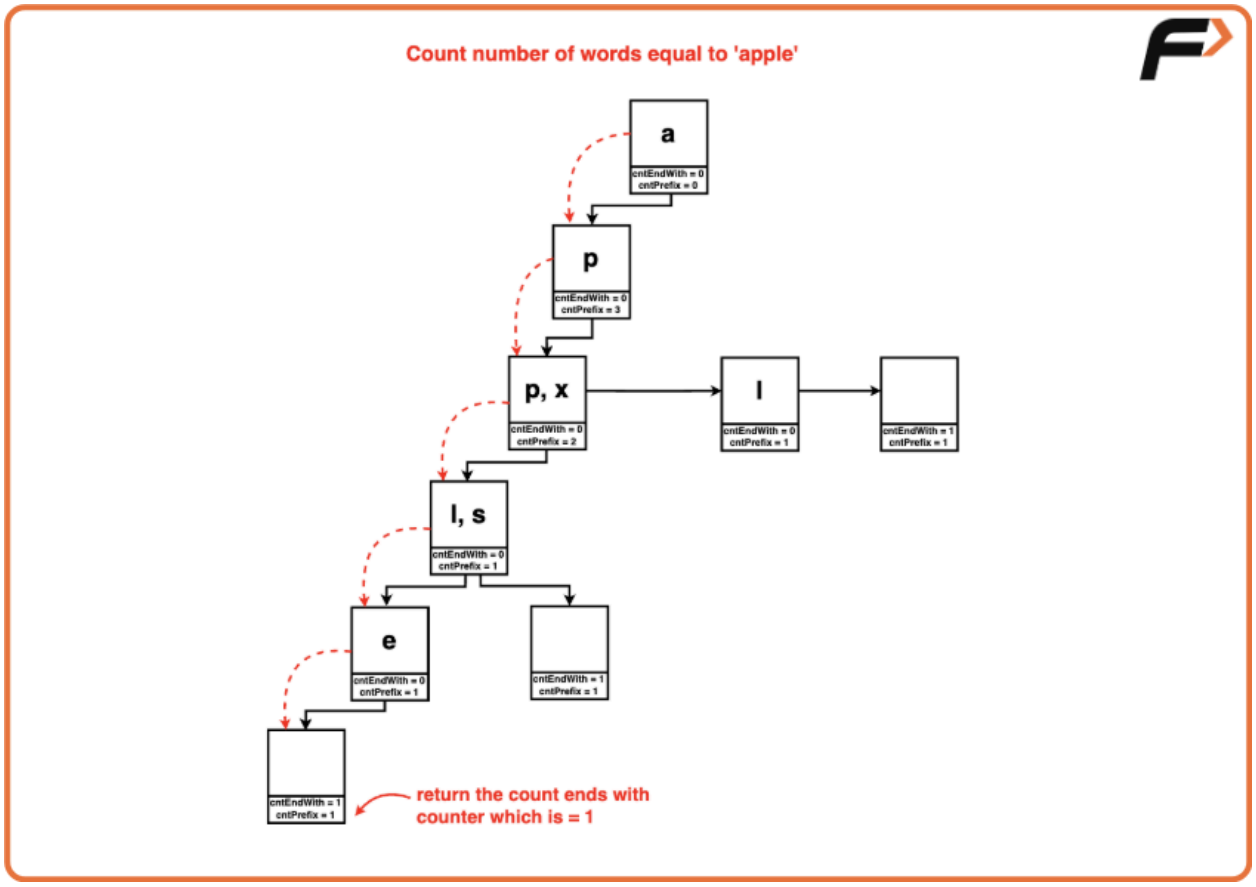
1️⃣ Insert(word)

- Traverse character by character
- Create nodes if missing
- **Increment `cntPrefix` for every visited node**
- At the last node, **increment `cntEndWith`**



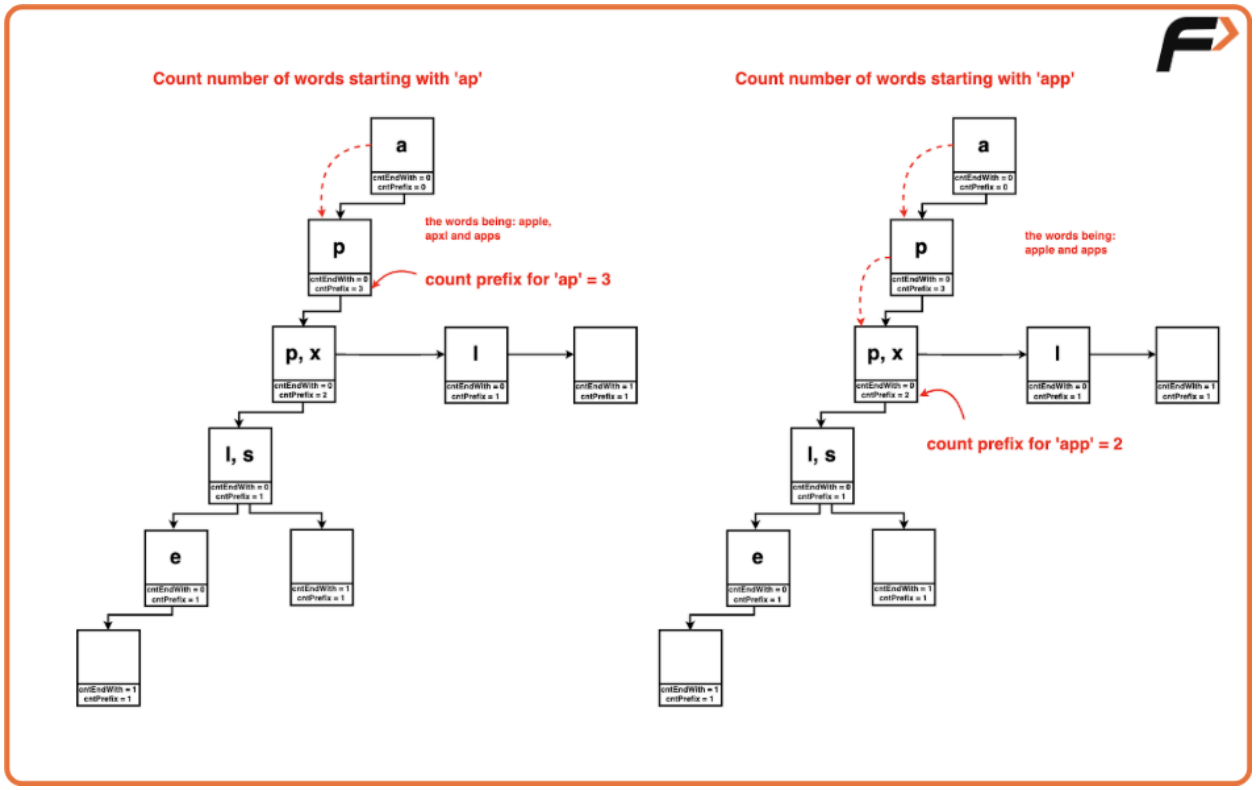
2️⃣ countWordsEqualTo(word)

- Traverse the word
- If path breaks → return `0`
- Otherwise → return `cntEndWith` of last node



3️⃣ countWordsStartingWith(prefix)

- Traverse the prefix
- If path breaks → return `0`
- Otherwise → return `cntPrefix` of last node

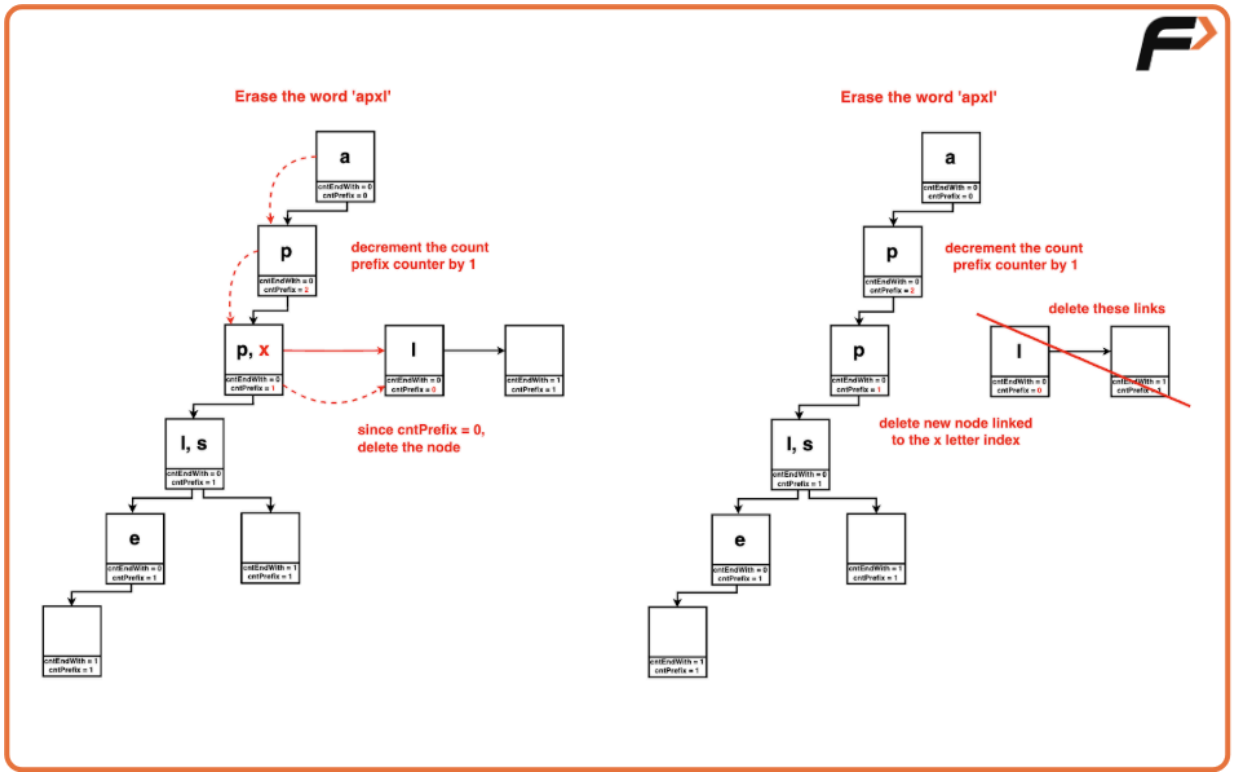


---

4 erase(word)

- Traverse the word
- **Decrement `cntPrefix` at every node**
- At the last node, **decrement `cntEndWith`**

We do **not delete nodes**, because other words may still depend on them.



```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    Node* links[26] = {nullptr};
    int cntEndWith = 0;   // words ending here
    int cntPrefix = 0;   // words passing through

    bool containsKey(char ch) {
        return links[ch - 'a'] != nullptr;
    }

    Node* get(char ch) {
        return links[ch - 'a'];
    }

    void put(char ch, Node* node) {
        links[ch - 'a'] = node;
    }
```

```
};

class Trie {
private:
    Node* root;

public:
    Trie() {
        root = new Node();
    }

    void insert(string word) {
        Node* node = root;
        for (char ch : word) {
            if (!node->containsKey(ch)) {
                node->put(ch, new Node());
            }
            node = node->get(ch);
            node->cntPrefix++;
        }
        node->cntEndWith++;
    }

    int countWordsEqualTo(string word) {
        Node* node = root;
        for (char ch : word) {
            if (!node->containsKey(ch)) return 0;
            node = node->get(ch);
        }
        return node->cntEndWith;
    }

    int countWordsStartingWith(string prefix) {
        Node* node = root;
        for (char ch : prefix) {
            if (!node->containsKey(ch)) return 0;
            node = node->get(ch);
        }
```

```
        return node->cntPrefix;
    }

    void erase(string word) {
        Node* node = root;
        for (char ch : word) {
            if (!node->containsKey(ch)) return;
            node = node->get(ch);
            node->cntPrefix--;
        }
        node->cntEndWith--;
    }
};

int main() {
    Trie trie;
    trie.insert("apple");
    trie.insert("apple");

    cout << trie.countWordsEqualTo("apple") << endl;     // 2
    cout << trie.countWordsStartingWith("app") << endl;  // 2

    trie.erase("apple");

    cout << trie.countWordsEqualTo("apple") << endl;     // 1
    cout << trie.countWordsStartingWith("app") << endl;  // 1
}
```

Time & Space Complexity

⏱ Time Complexity

For all operations:

- **O(L)** where L = length of the word or prefix

💾 Space Complexity

- **O(total characters inserted)**
  Worst case: every character creates a new Trie node

3. Longest Valid Word with All Prefixes

Given an array of strings, you have to find the **longest string** such that **all of its prefixes are also present** in the array.
If more than one string has the same maximum length, return the **lexicographically smallest** one.

A prefix means a substring starting from the first character. The word itself is also considered a prefix.

Example
arr = ["ab","a","abc","abd"]
Both "abc" and "abd" are valid because their prefixes exist.
"abc" → "a","ab","abc"
"abd" → "a","ab","abd"
Both have same length, so lexicographically smaller "abc" is the answer.

Approach 1

Using Sorting and Binary Search

Algorithm

1. Sort the array lexicographically so binary search can be used.
2. For each word in the array:
    - Build its prefixes one by one.

        - For every prefix, check if it exists in the array using binary search.
    3. If all prefixes exist, the word is valid.
    4. Update the result if:
        - The word is longer than current result, or
        - Length is same but word is lexicographically smaller.

Code

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

string longestString(vector<string>& arr) {
    sort(arr.begin(), arr.end());
    string result;

    for (string& word : arr) {
        bool isValid = true;
        string prefix;

        for (char ch : word) {
            prefix += ch;
            if (!binary_search(arr.begin(), arr.end(), prefix)) {
                isValid = false;
                break;
            }
        }

        if (isValid &&
            (word.size() > result.size() ||
            (word.size() == result.size() && word < result))) {
            result = word;
        }
    }
    return result;
}

int main() {
    vector<string> arr = {"ab","a","abc","abd"};
    cout << longestString(arr);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(n * k² * log n)
  For each word (n), we generate k prefixes, and each binary search takes O(k log n) due to string comparison.
- **Space Complexity:** O(1)
  No extra data structure used apart from variables.

Approach 2

Rabin-Karp Based Prefix Validation

Algorithm

1. Use double hashing to reduce collision chances.
2. Compute hash pairs for all words and store them in a set.
3. For each word:
    - Generate prefix hashes incrementally.
    - Check if each prefix hash exists in the set.
4. If all prefixes exist, update the result using length and lexicographical order.

Code

```
#include <iostream>
#include <vector>
#include <set>
using namespace std;

const int base1 = 31, mod1 = 1e9 + 7;
const int base2 = 37, mod2 = 1e9 + 9;

vector<int> computeHash(string& s) {
   int h1 = 0, h2 = 0, p1 = 1, p2 = 1;
   for (char ch : s) {
      int val = ch - 'a' + 1;
      h1 = (h1 + val * 1LL * p1) % mod1;
      h2 = (h2 + val * 1LL * p2) % mod2;
      p1 = (p1 * 1LL * base1) % mod1;
      p2 = (p2 * 1LL * base2) % mod2;
   }
   return {h1,h2};
}

bool allPrefixExist(string& word, set<vector<int>>& hashSet) {
```

```
    int h1 = 0, h2 = 0, p1 = 1, p2 = 1;
    for (char ch : word) {
        int val = ch - 'a' + 1;
        h1 = (h1 + val * 1LL * p1) % mod1;
        h2 = (h2 + val * 1LL * p2) % mod2;
        if (hashSet.find({h1,h2}) == hashSet.end())
            return false;
        p1 = (p1 * 1LL * base1) % mod1;
        p2 = (p2 * 1LL * base2) % mod2;
    }
    return true;
}

string longestString(vector<string>& arr) {
    set<vector<int>> hashSet;
    for (string& word : arr)
        hashSet.insert(computeHash(word));

    string result;
    for (string& word : arr) {
        if (allPrefixExist(word, hashSet)) {
            if (word.size() > result.size() ||
               (word.size() == result.size() && word < result)) {
                result = word;
            }
        }
    }
    return result;
}

int main() {
    vector<string> arr = {"ab","a","abc","abd"};
    cout << longestString(arr);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(n * k * log n)
  Each prefix check involves set lookup.
- **Space Complexity:** O(n)
  Hash values of all words are stored.

Approach 3

Trie-Based Prefix Validation

Algorithm

1. Create a Trie node with 26 children and an `isEnd` flag.
2. Insert all words into the Trie.
3. For each word:
    - Traverse the Trie character by character.
    - At every step, ensure the node exists and `isEnd` is true.
4. Update the result using length and lexicographical comparison.

Code

```
#include <iostream>
#include <vector>
using namespace std;

struct TrieNode {
    TrieNode* children[26];
    bool isEnd;
    TrieNode() {
        isEnd = false;
        for (int i = 0; i < 26; i++) children[i] = nullptr;
    }
};

class Trie {
public:
    TrieNode* root;
    Trie() {
        root = new TrieNode();
    }

    void insert(string& word) {
        TrieNode* node = root;
        for (char ch : word) {
            int idx = ch - 'a';
            if (!node->children[idx])
                node->children[idx] = new TrieNode();
            node = node->children[idx];
        }
        node->isEnd = true;
    }
```

```
    bool allPrefixesExist(string& word) {
        TrieNode* node = root;
        for (char ch : word) {
            int idx = ch - 'a';
            node = node->children[idx];
            if (!node || !node->isEnd)
                return false;
        }
        return true;
    }
};

string longestString(vector<string>& arr) {
    Trie trie;
    for (string& word : arr)
        trie.insert(word);

    string result;
    for (string& word : arr) {
        if (trie.allPrefixesExist(word)) {
            if (word.size() > result.size() ||
               (word.size() == result.size() && word < result)) {
                result = word;
            }
        }
    }
    return result;
}

int main() {
    vector<string> arr = {"ab","a","abc","abd"};
    cout << longestString(arr);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(n * k)
  Trie insertion and prefix checking both depend on word length.
- **Space Complexity:** O(n * k)
  Trie stores all characters of all words.

4. Number of Distinct Substrings in a String Using Trie

Given a string S, find the **number of distinct substrings**, including the **empty substring**.
A substring is formed by removing zero or more characters from the **start and end** of the string.

Two substrings are different if their lengths are different or at least one character differs at the same index.

Example
s = "aba"
Distinct substrings are: `"a","ab","aba","b","ba",""`
Output = 6

s = "abc"
Distinct substrings are: `"a","ab","abc","b","bc","c",""`
Output = 7

Approach 1

Brute-Force Using Set

Algorithm

1. Create an empty set to store distinct substrings.
2. Fix a starting index `i` from `0` to `n-1`.
3. From index `i`, build substrings by extending till the end using index `j`.
4. Insert each generated substring into the set.
5. After all substrings are inserted, count the size of the set.
6. Add 1 to include the empty substring.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    set<string> countDistinctSubstrings(const string& s) {
        set<string> st;
        int n = s.length();
```

```
        for (int i = 0; i < n; i++) {
            string str = "";
            for (int j = i; j < n; j++) {
                str += s[j];
                st.insert(str);
            }
        }
        return st;
    }
};

int main() {
    string s = "striver";
    Solution sol;

    set<string> substrings = sol.countDistinctSubstrings(s);
    int count = substrings.size();

    cout << "Number of distinct substrings: " << count + 1 << endl;
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(N²)
  All possible substrings are generated using two loops.
- **Space Complexity:** O(N²)
  The set can store up to N² substrings in the worst case.

Approach 2

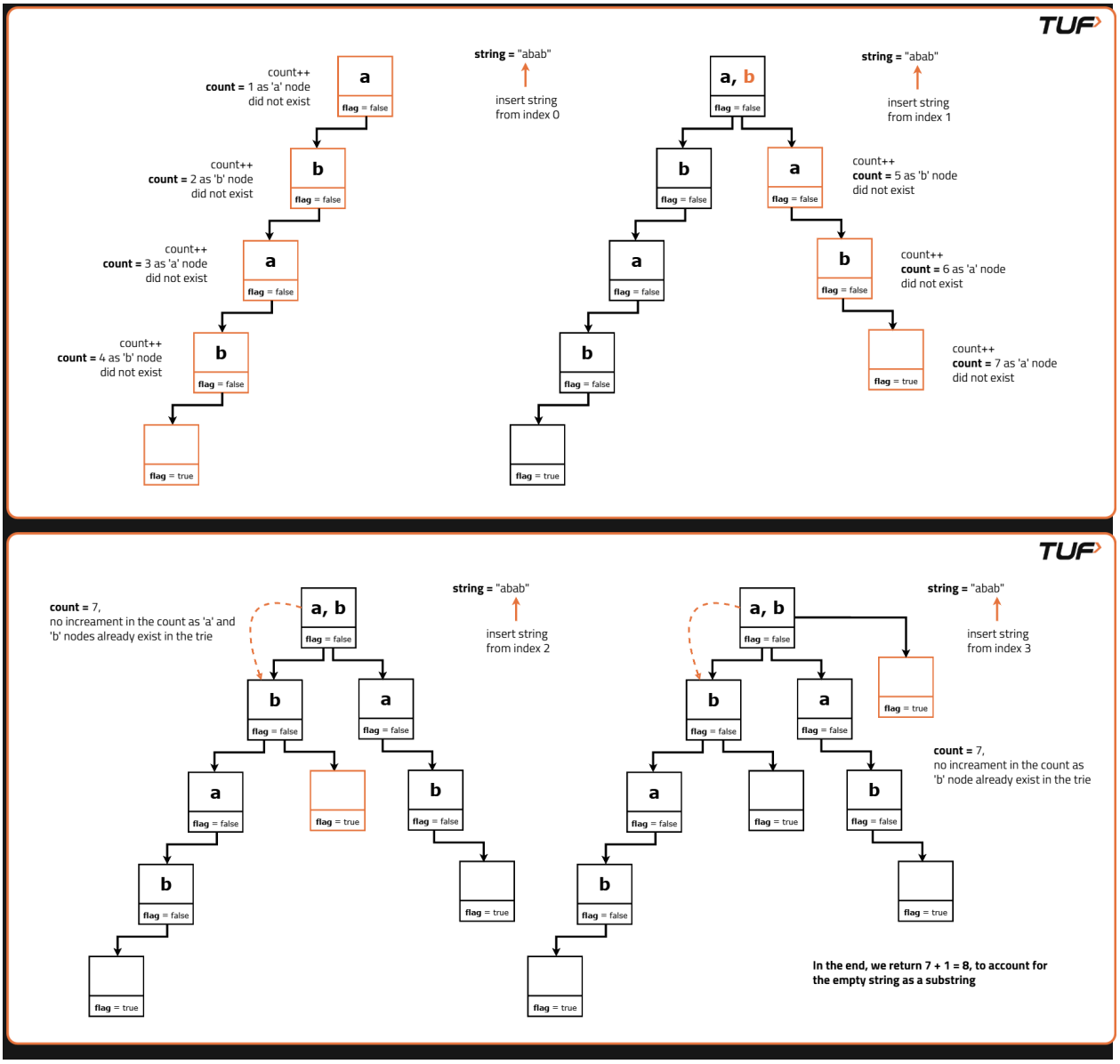
Trie-Based Optimal Approach

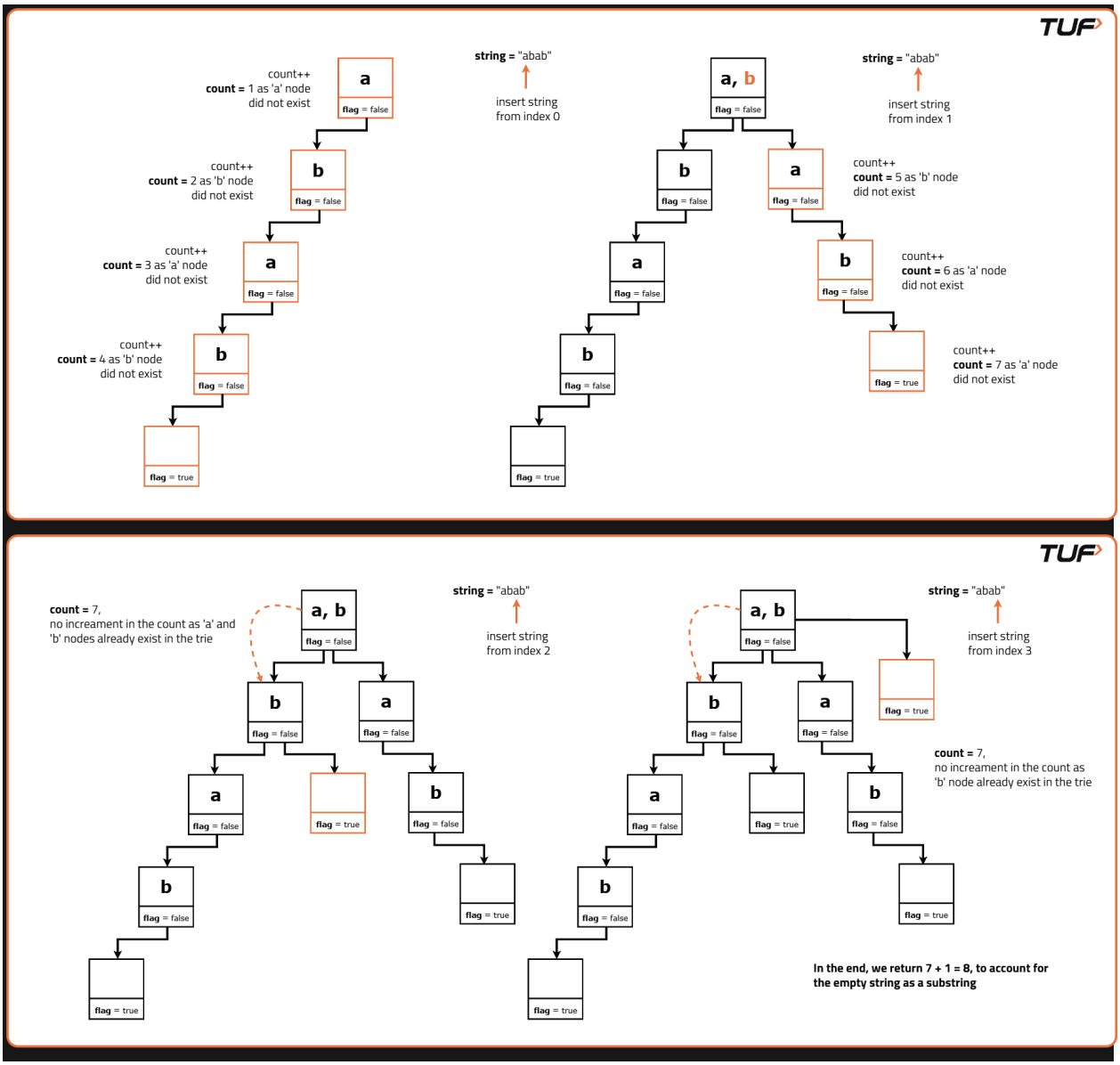
Algorithm

1. Create a Trie with a root node.
2. Initialize a counter to count distinct substrings.
3. For each starting index `i`:
    - Start from the root node.
    - For every character `j` from `i` to end:

- If the character path does not exist in the Trie, create it and increment the counter.
- Move to the next Trie node.

4. Each new Trie node represents a **new distinct substring**.
5. Add 1 at the end to count the empty substring.





Code

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    Node* links[26];
    bool flag = false;

    bool containsKey(char ch) {
        return links[ch - 'a'] != NULL;
    }
```

```
    Node* get(char ch) {
        return links[ch - 'a'];
    }

    void put(char ch, Node* node) {
        links[ch - 'a'] = node;
    }
};

class Solution {
public:
    int countDistinctSubstrings(string &s) {
        Node* root = new Node();
        int cnt = 0;
        int n = s.size();

        for (int i = 0; i < n; i++) {
            Node* node = root;
            for (int j = i; j < n; j++) {
                if (!node->containsKey(s[j])) {
                    node->put(s[j], new Node());
                    cnt++;
                }
                node = node->get(s[j]);
            }
        }
        return cnt + 1;
    }
};

int main() {
    string s = "striver";
    Solution sol;
    cout << "Number of distinct substrings: " << sol.countDistinctSubstrings(s) << endl;
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N^2)$
  All substrings are processed once, and Trie insertion is O(1) per character.
- **Space Complexity:** $O(N^2)$
  Trie stores nodes for all unique substrings.

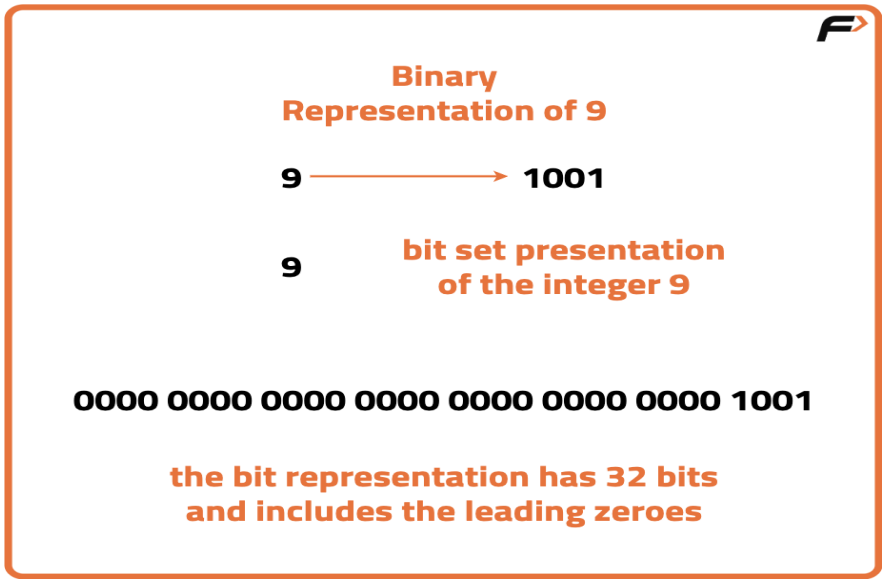
5. Bit PreRequisites for TRIE Problems

Up until now we have looked at Trie and the various functions that can be performed on it like searching, insertion and fniding prefix. An important application of bitwise operations in tries is XOR (exclusive OR). XOR is commonly used in tries to efficiently perform operations like bitwise toggling or finding the XOR of elements. This operation is particularly useful in scenarios where we need to compare or manipulate binary representations of data.

Basics of Bit Manipulation

Binary Representation

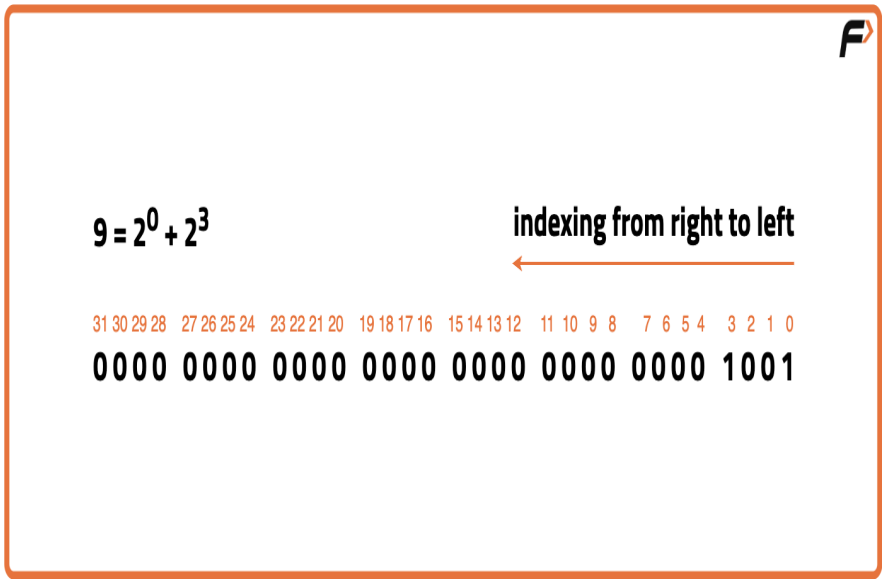
When representing the integer 9 in binary, it's represented as “1001”. However, in Trie data structures, each bit position represents a decision point in the trie, so the entire bit set is stored ie. “0000 … 1001”, including leading zeros.



Therefore, in a trie, the binary representation of 9 would indeed be stored as "0000 …. 1001" comprising of a total of 32 bits, where each bit position represents a node in the trie, regardless of the leading zeroes.

Indexing in Binary Representation

In binary representation, indexing typically starts from the rightmost bit, also known as the least significant bit (LSB). This means that the rightmost bit has an index of 0, and the index increases as you move towards the left, with the leftmost bit having the highest index. This convention is consistent with the positional numbering system in binary, where each bit represents a power of 2, starting from 2^0 (1) on the right and increasing by powers of 2 as you move left. For example, let's consider the binary representation of the integer 9: "0000 …. 1001". In this representation:



The right bit (the first bit from the right) has an index of 0. The second rightmost bit has an index of 1. This pattern continues, with each subsequent bit to the right having an index one higher than the previous bit. This indexing convention is essential for various bitwise operations, including those used in tries, where the position of each bit matters for traversal and manipulation of the data structure.

XOR Operation

The XOR (exclusive OR) operation is a fundamental bitwise operation that compares corresponding bits of two operands. The result of the XOR operation is 1 if the bits are different and 0 if they are the same.

Here's how the XOR operation works:

- If both bits are 0, the result is 0.
- If both bits are 1, the result is 0.
- If one bit is 0 and the other is 1, the result is 1.

In other words, the XOR operation yields a 1 only if the bits being compared are different; otherwise, it yields a 0.

Here's the XOR truth table:

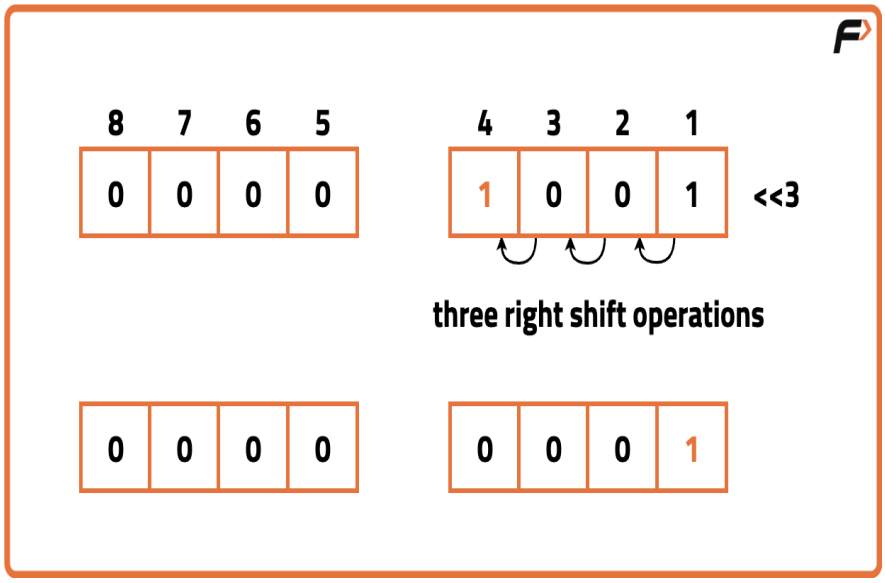
Input 1	Input 2	Output
0	0	0
0	1	1
1	0	1
1	1	0

Important Points to Note are:

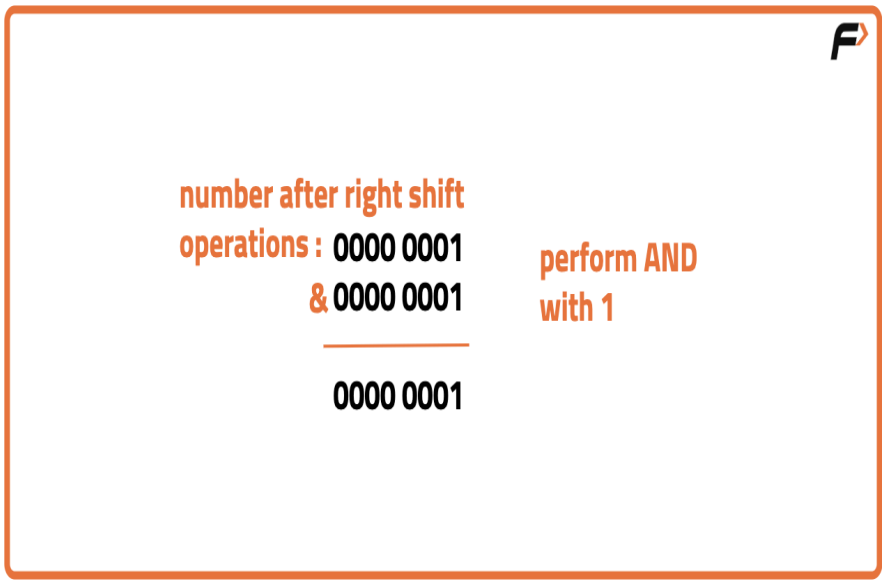
- If there are an even number of 1s (e.g., 0, 2, 4...), the XOR operation will result in 0. This is because for every 1, there must be another 1 paired with it to produce a 0 output (e.g., 1 XOR 1 = 0).
- If there are an odd number of 1s (e.g., 1, 3, 5...), the XOR operation will result in 1. This is because there will always be one unpaired 1 left, resulting in a 1 output (e.g., 1 XOR 1 XOR 1 = 1).

How to Check is a Bit is set or not

To check if a particular bit is set (i.e., has a value of 1) or not in a binary number, we use a bitwise AND operation.

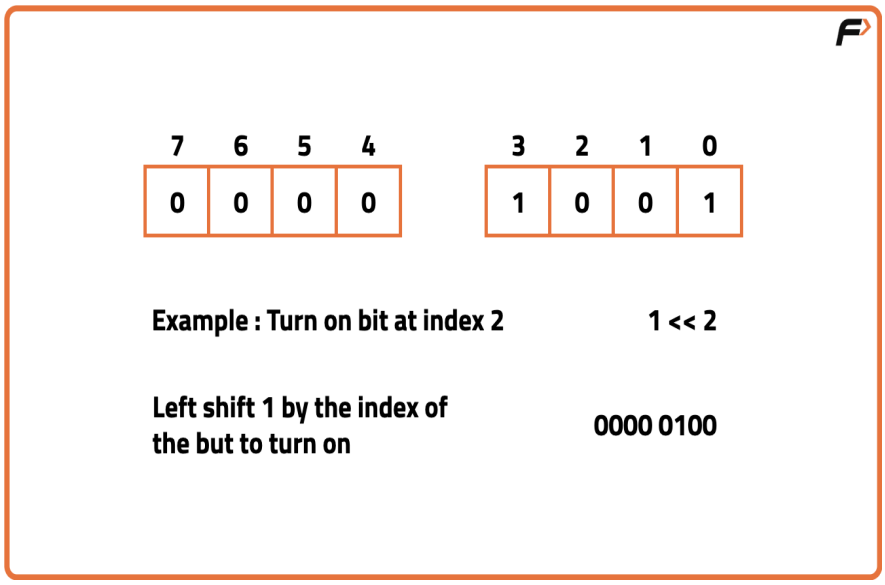


- Right shift the number by the index of the bit you want to check.
- Perform a bitwise AND operation with 1.
- Check if the result is equal to 1.

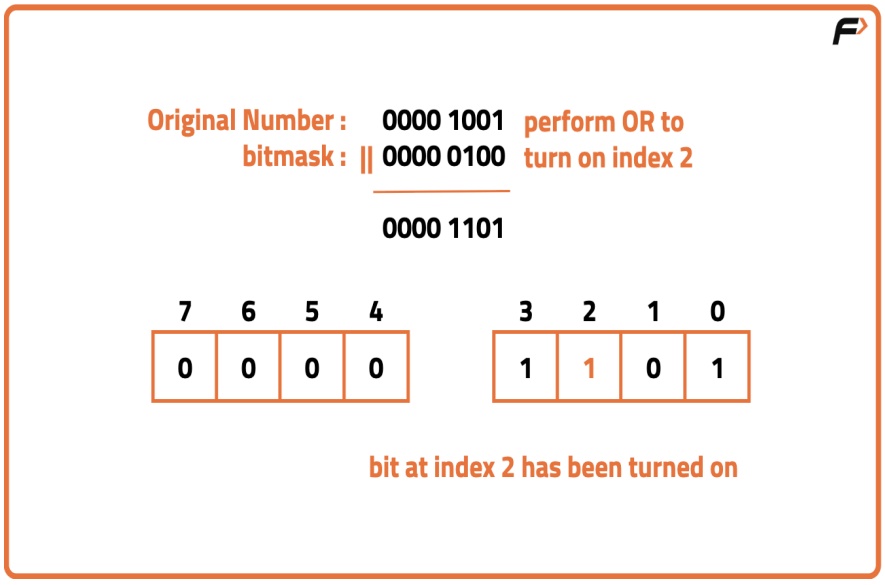


How to Turn On a Bit

To "turn on" a bit means to set it to the value of 1 within a binary number. This operation involves modifying the binary representation of a number such that the targeted bit becomes 1 while leaving other bits unchanged. Turning on a bit is often done using bitwise OR operation.



- Create a bitmask with the bit you want to turn on set to 1 and all other bits set to 0.
- Perform a bitwise OR operation between the original number and the bitmask.



6. Maximum XOR of Two Numbers in an Array

Given an integer array `nums`, you have to find the **maximum XOR value** that can be obtained using `nums[i] XOR nums[j]` where `0 ≤ i ≤ j < n`.

XOR (exclusive OR) gives 1 when the two bits are different and `0` when they are the same.
To maximize XOR, we try to make as many bit positions different as possible between two numbers.

Example
nums = [3, 9, 10, 5, 1]
Binary:

- 10 → 1010
- 5 → 0101
  XOR → 1111 = 15

Approach

Using Trie (Binary Trie)

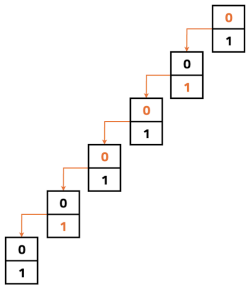
Algorithm

1. Create a Trie where each node has two links: `0` and 1.
2. Insert every number into the Trie by processing its bits from **most significant bit (31)** to **least significant bit (0)**.
3. To find maximum XOR for a number:
   - Traverse the Trie again from bit 31 to 0.
   - For each bit, try to move to the **opposite bit** (1 − bit).
   - If the opposite bit exists, set the corresponding bit in XOR result.
   - Otherwise, move to the same bit.
4. Repeat this for all numbers and keep track of the maximum XOR value.

This works because choosing opposite bits at higher positions gives a larger XOR value.

Input Array : [9, 8, 7, 5, 4] Increasing 9 into the Trie

9 = 01001

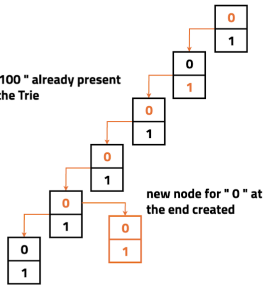


Input Array : [9, 8, 7, 5, 4] Increasing 9 into the Trie

8 = 01000

" 0100 " already present in the Trie

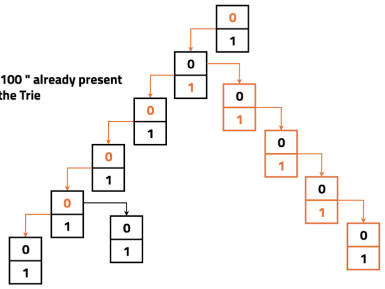
new node for " 0 " at the end created



Input Array : [9, 8, 7, 5, 4] Increasing 9 into the Trie

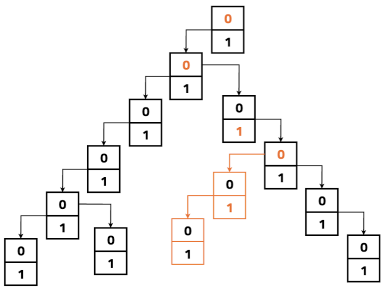
7 = 00111

" 0100 " already present in the Trie



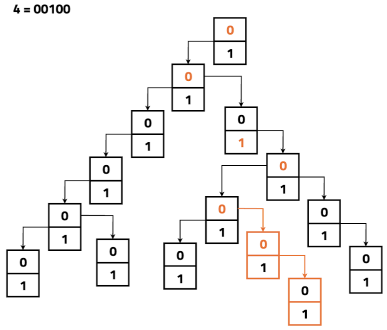
Input Array : [9, 8, 7, 5, 4] Increasing 5 into the Trie

5 = 00101



Input Array : [9, 8, 7, 5, 4] Increasing 4 into the Trie

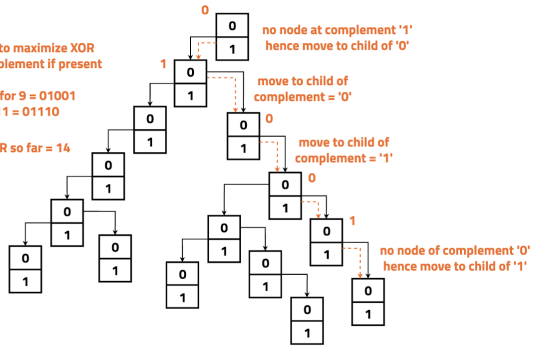
4 = 00100

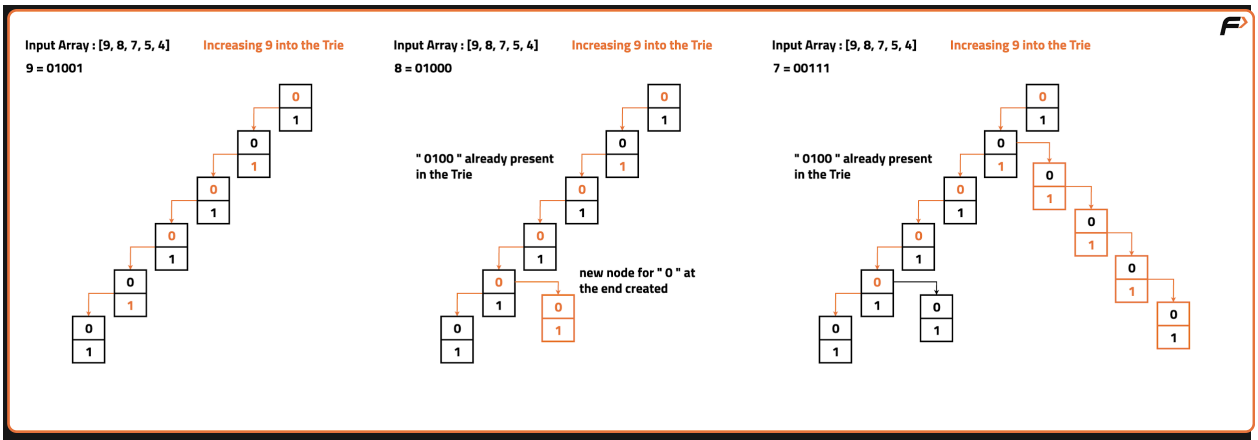


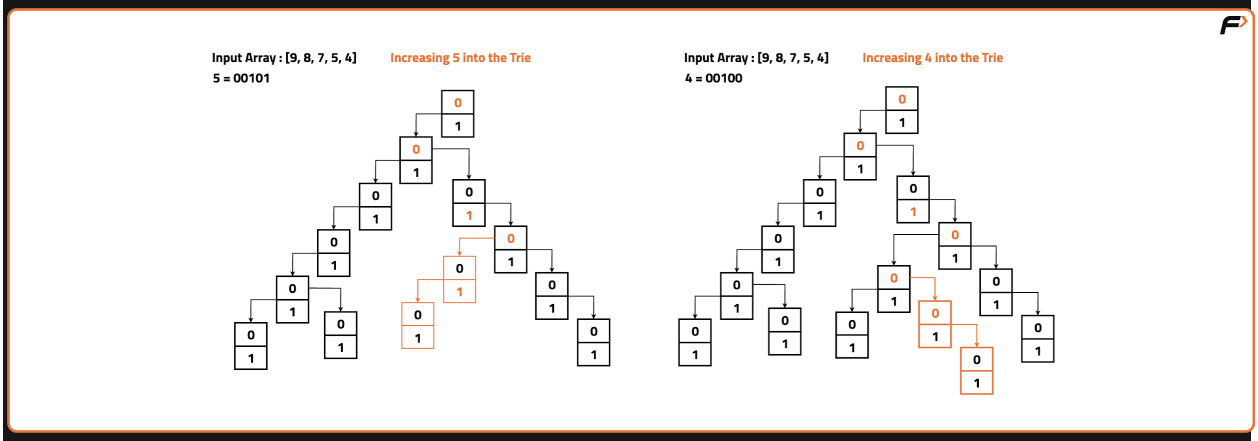
Iterating over 9 = 01001 to maximize XOR
At each node find the complement if present

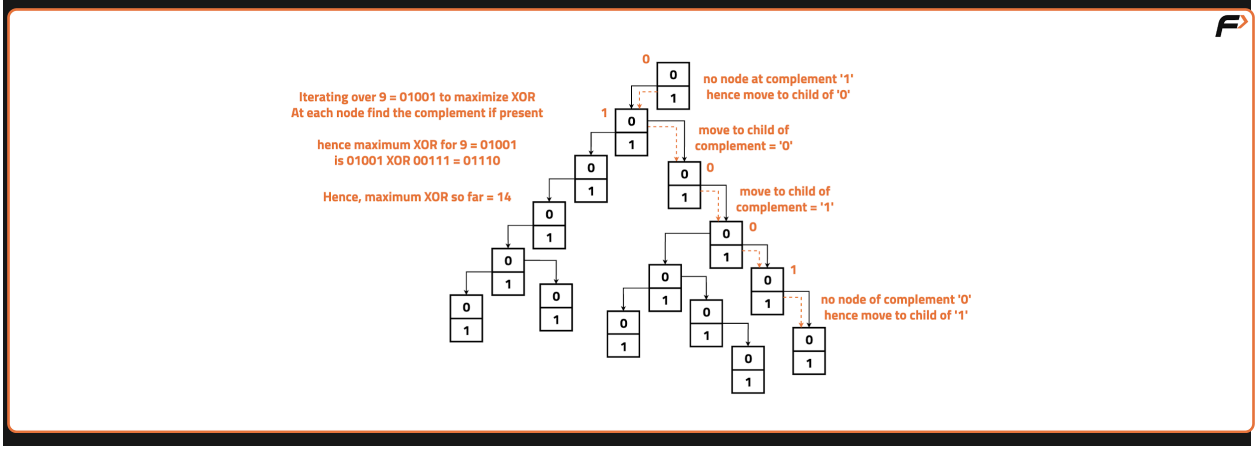
hence maximum XOR for 9 = 01001
is 01001 XOR 00111 = 01110

Hence, maximum XOR so far = 14










```
#include <bits/stdc++.h>

using namespace std;


class Node {
```

```
public:
    Node* links[2];

    bool containsKey(int bit) {
        return links[bit] != NULL;
    }

    Node* get(int bit) {
        return links[bit];
    }

    void put(int bit, Node* node) {
        links[bit] = node;
    }
};

class Solution {
public:
    Node* root;

    Solution() {
        root = new Node();
    }
```

```
void insert(int num) {
    Node* node = root;
    for (int i = 31; i >= 0; i--) {
        int bit = (num >> i) & 1;
        if (!node->containsKey(bit)) {
            node->put(bit, new Node());
        }
        node = node->get(bit);
    }
}

int getMaxXOR(int num) {
    Node* node = root;
    int maxXor = 0;

    for (int i = 31; i >= 0; i--) {
        int bit = (num >> i) & 1;
        if (node->containsKey(1 - bit)) {
            maxXor |= (1 << i);
            node = node->get(1 - bit);
        } else {
            node = node->get(bit);
        }
    }
```

```
        return maxXor;
    }

    int findMaximumXOR(vector<int>& nums) {
        for (int num : nums) {
            insert(num);
        }

        int maxResult = 0;
        for (int num : nums) {
            maxResult = max(maxResult, getMaxXOR(num));
        }
        return maxResult;
    }
};

int main() {
    vector<int> nums = {3, 9, 10, 5, 1};
    Solution sol;
    cout << sol.findMaximumXOR(nums);
    return 0;
}
```

- **Time Complexity:** O(N)
  Each number is inserted and queried in the Trie using 32 bits, which is constant.
- **Space Complexity:** O(N)
  Trie stores up to 32 nodes per number in the worst case.

7. Maximum Xor Queries | Trie

You are given an array of **non-negative integers** and a list of queries.
Each query is of the form **[Xi, Ai]**.

For each query, you have to find the **maximum value of (Xi XOR num)** such that:

- `num` is taken from the array
- `num ≤ Ai`

If **no number in the array is ≤ Ai**, return **-1** for that query.

Example
Array = [3, 10, 5, 25, 2, 8]
Queries = [(0,1), (1,2), (3,3)]

For (0,1): no element ≤ 1 → answer = -1
For (1,2): only element ≤ 2 is 2 → 1 XOR 2 = 3
For (3,3): elements ≤ 3 are 2 and 3 → max(3 XOR 2, 3 XOR 3) = 1

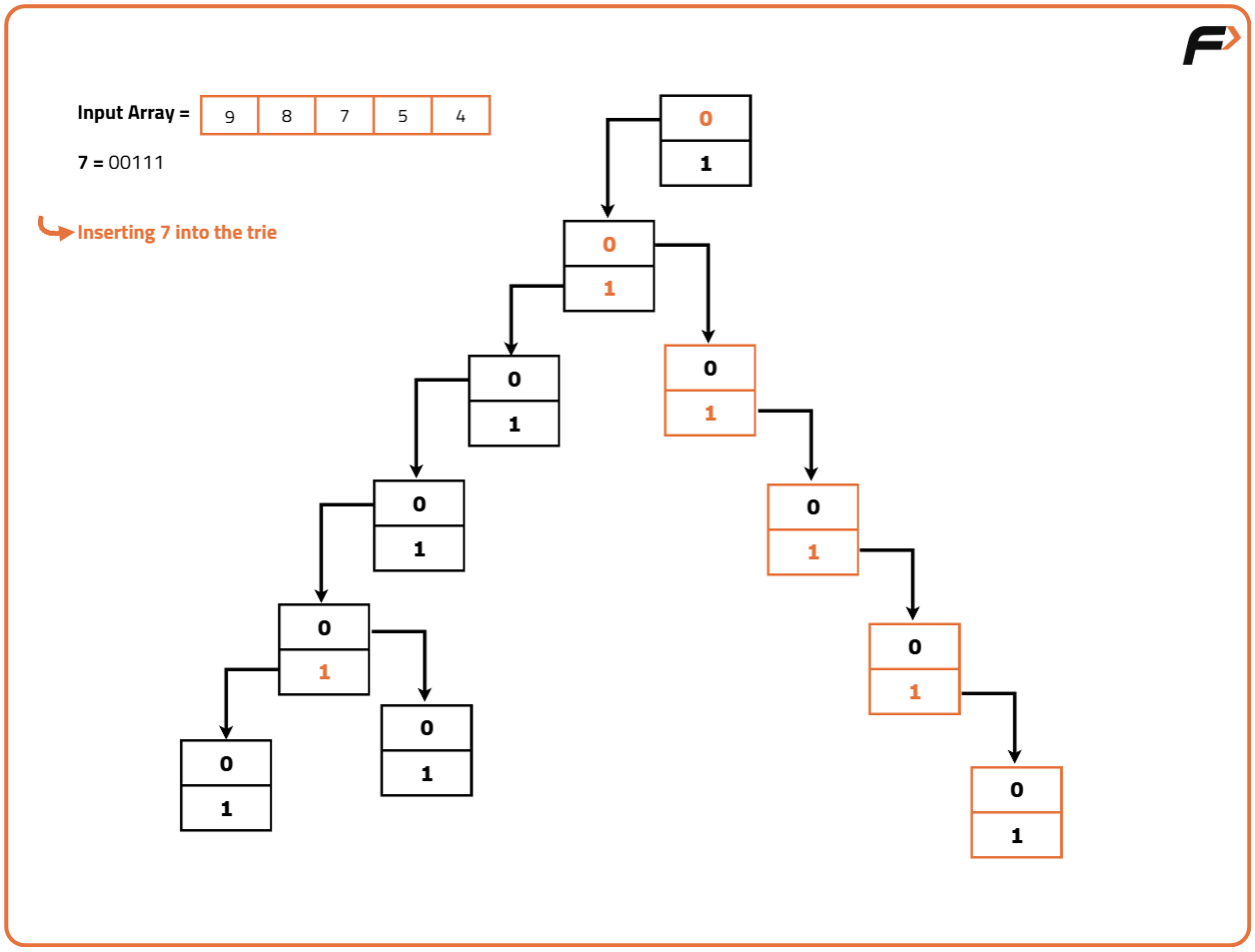
Approach

Trie + Offline Query Processing

Algorithm

1. XOR gives maximum value when bits are different.
2. To maximize XOR efficiently, we use a **binary Trie** where:
    - Each node has two links: `0` and `1`
    - Each path represents the binary form of a number
3. Instead of inserting all array elements at once:
    - Sort the array
    - Sort queries by `Ai`
4. Process queries one by one:
    - Insert all array elements **≤ current Ai** into the Trie
    - If Trie is empty, answer is `-1`
    - Otherwise, traverse the Trie to find maximum XOR for `Xi`
5. Store answers using original query indices.

This avoids unnecessary insertions and keeps operations efficient.




```
#include <bits/stdc++.h>
using namespace std;

// Trie node
struct Node {
    Node* links[2];

    bool containsKey(int bit) {
        return links[bit] != NULL;
    }
```

```
    Node* get(int bit) {
        return links[bit];
    }

    void put(int bit, Node* node) {
        links[bit] = node;
    }
};

// Trie class
class Trie {
    Node* root;

public:
    Trie() {
        root = new Node();
    }

    void insert(int num) {
        Node* node = root;
        for (int i = 31; i >= 0; i--) {
            int bit = (num >> i) & 1;
            if (!node->containsKey(bit)) {
```

```
            node->put(bit, new Node());
        }
        node = node->get(bit);
    }
  }

  int findMax(int num) {
    Node* node = root;
    int maxXor = 0;

    for (int i = 31; i >= 0; i--) {
        int bit = (num >> i) & 1;
        if (node->containsKey(1 - bit)) {
            maxXor |= (1 << i);
            node = node->get(1 - bit);
        } else {
            node = node->get(bit);
        }
    }
    return maxXor;
  }
};

class Solution {
```

```
public:
    vector<int> maxXorQueries(vector<int>& arr,
                              vector<vector<int>>& queries) {

        vector<int> ans(queries.size(), 0);
        vector<pair<int, pair<int,int>>> offlineQueries;

        sort(arr.begin(), arr.end());

        int idx = 0;
        for (auto& q : queries) {
            offlineQueries.push_back({q[1], {q[0], idx++}});
        }

        sort(offlineQueries.begin(), offlineQueries.end());

        Trie trie;
        int i = 0, n = arr.size();

        for (auto& q : offlineQueries) {
            int m = q.first;
            int x = q.second.first;
            int index = q.second.second;
```

```
            while (i < n && arr[i] <= m) {
                trie.insert(arr[i]);
                i++;
            }

            if (i == 0)
                ans[index] = -1;
            else
                ans[index] = trie.findMax(x);
        }
        return ans;
    }
};

int main() {
    vector<int> arr = {3, 10, 5, 25, 2, 8};
    vector<vector<int>> queries = {{0,1}, {1,2}, {3,3}};

    Solution obj;
    vector<int> result = obj.maxXorQueries(arr, queries);

    for (int x : result)
        cout << x << " ";
    return 0;
```

}

Complexity Analysis

- **Time Complexity:**
  $O(32 \cdot N + Q \cdot \log Q + 32 \cdot Q)$
  - 32 bits for each insertion and query
  - Sorting queries costs $Q \cdot \log Q$
- **Space Complexity:**
  $O(32 \cdot N + Q)$
  - Trie stores bits of array elements
  - Extra space for offline queries and answers